

Plan de Pruebas – QA Proyecto ScoutApp

1. Introducción

Este documento establece la estrategia de Aseguramiento de la Calidad (QA) para el proyecto *ScoutApp*.

El propósito es garantizar que los módulos críticos funcionen de forma correcta, segura y aislada por tenant, utilizando siempre datos ficticios para evitar riesgos de privacidad.

El trabajo de QA se realizará en ciclos iterativos cortos, acompañando al desarrollo en cada historia de usuario. El foco estará en:

- **Pruebas funcionales:** verificar que cada módulo cumpla los criterios de aceptación.
- **Smoke testing:** validar el conjunto mínimo de pruebas antes de cada despliegue.
- **Seguridad multi-tenant:** garantizar que los datos de un grupo no sean visibles desde otro.

2. Alcance de las Pruebas

2.1 Módulos incluidos en esta fase

- **Login y autenticación** (incluye validación multi-tenant).
- **Gestión de miembros:** operaciones CRUD básicas.
- **Gestión de eventos:** listar, crear, autorizar.
- **Roles y permisos:** validación de accesos según perfil (líder, miembro, comité).

2.2 Módulos excluidos (fuera de alcance inicial)

- **Pruebas de rendimiento** (carga, estrés).
- **Integración con servicios externos** (futuros).
- **Validaciones móviles nativas** (responsividad básica sí será observada).

Nota: El alcance se centrará en asegurar la funcionalidad crítica para el uso diario del sistema y en los aspectos de seguridad de datos.

3. Estrategia de Pruebas

Las pruebas se dividen en dos niveles principales:

- **Pruebas funcionales**
 - Se aplican a cada historia de usuario.
 - Validan que los criterios de aceptación estén cumplidos.
 - Cubren flujos completos: creación de un miembro, autorización de un evento, asignación de roles.
- **Pruebas no funcionales (nivel básico)**
 - Seguridad y privacidad (no exponer datos sensibles).
 - Aislamiento multi-tenant (un grupo no puede ver datos de otro).

3.1 Priorización

Se usará el criterio “**riesgo × impacto**”:

- **Alta prioridad (P1 – Bloquea operación o compromete datos/privacidad):**
Casos donde un fallo impide el funcionamiento del sistema o pone en riesgo la seguridad/privacidad de la información.
Ejemplo: un usuario no puede iniciar sesión, o un tenant accede a datos de otro grupo.
- **Media prioridad (P2 – Afecta tareas relevantes pero con alternativas):**
Casos donde el error impacta en procesos importantes, pero el sistema aún se puede usar mediante una vía alternativa o con limitaciones.
Ejemplo: un rol con permisos limitados no recibe un mensaje claro de restricción, pero aun así se le bloquea el acceso.
- **Baja prioridad (P3 – Usabilidad/estética sin impacto funcional):**
Casos que no afectan el flujo de trabajo ni la seguridad, relacionados con la experiencia de usuario, apariencia o mensajes secundarios.
Ejemplo: un error tipográfico en la interfaz o un color distinto al esperado.

4. Ambientes de Prueba

- **Local (feature):**
 - Uso de APIs mockeadas y base de datos local con datos ficticios.
 - Sirve para validar rápidamente historias nuevas.
- **Staging (develop):**
 - Entorno similar a producción.
 - Pruebas de integración y smoke testing.
 - **HTTPS obligatorio** para todas las URLs.
- **Master(producción):**
 - Manejada por el docente.
 - QA solo validará con **datos ficticios** después de cada despliegue.

5. “Listo para probar” – Readliness Checklist

Una historia **solo puede ser probada** cuando cumple todos los siguientes puntos:

- **Historia y Criterios de Aceptación** claros y documentados en el issue/PR.
- **Criterios entendidos por QA**, con ejemplos de los flujos esperados.
- **Contrato de API publicado**: ruta, método, request/response y códigos de error documentados.
- **Colección Postman y entorno actualizados** (local/staging) con los endpoints nuevos.
- **Datos ficticios configurados** en BD o en entorno de prueba.
- **Logs útiles y anónimos** (sin PII), con manejo visible de errores en la UI.
- No existen **bloqueos técnicos** (ej. ambiente caído, falta de credenciales).

Nota: Si alguno de estos puntos no se cumple, QA no comienza la validación de esa historia.

6. Definición de Hecho (DoD) con QA

Una historia se considera **finalizada** y lista para cerrar cuando:

1. Cumple con todos los **criterios de aceptación**.
2. Ha pasado la **suite de smoke tests** definida.
3. No presenta **defectos críticos** (severidad alta).
4. Usa **exclusivamente datos ficticios** en todas las pruebas.
5. Supera las pruebas de **aislamiento multi-tenant**
6. Se ha dejado **evidencia documentada** (capturas, reportes de Postman, observaciones en el issue/PR).

7. Privacidad y Datos

Todas las pruebas se realizarán únicamente con **datos ficticios o sintéticos** (ej. nombres y documentos inventados). Nunca se usarán datos reales de jóvenes o familias. Las evidencias (capturas, reportes) deberán estar libres de información sensible, y los módulos sensibles como **ficha médica** y **aportes** se validarán únicamente con datos simulados.

Casos de Prueba Iniciales

Módulo 1: Miembros / Ramas

Caso: Edición de miembro existente

- Dado: un miembro ya registrado
- Cuando: un usuario autorizado edita su información
- Entonces: los cambios se guardan y se reflejan en el historial del miembro
- **Endpoint:** PUT /members/{id}
- **Respuestas:**
 - **200 OK:** Actualiza datos del miembro. Ej.: { id, ...actualizado }.
 - **404 Not Found:** El miembro no existe en el tenant.

Caso: Asignación a rama según edad

- Dado: un nuevo miembro con edad válida para una rama
- Cuando: se le asigna a la rama correspondiente
- Entonces: queda registrado en esa rama con su fecha de ingreso
- **Endpoint:** POST /members/{id}/assign-branch
- **Respuestas:**
 - **200 OK:** Asigna rama. Ej.: { branch, sinceDate }.
 - **400/422 Bad Request:** La rama no corresponde por edad/reglas.

Caso: Movimiento entre ramas

- Dado: un miembro ya asignado a una rama
- Cuando: se autoriza su cambio a otra rama
- Entonces: se actualiza la fecha de salida y entrada en el historial del miembro
- **Endpoint:** POST /members/{id}/move-branch
- **Respuestas:**
 - **200 OK:** Registra salida/entrada en historial. Ej.: { history:[...] }.

Caso: Generación de reportes de miembros

- Dado: un Jefe/Subjefe de Grupo
- Cuando: genera un reporte por rama y estado (activo, baja, en espera)
- Entonces: el sistema muestra el listado con totales diferenciados y permite exportar a PDF/Excel
- **Endpoint:** GET /members/reports?branch=&status=

- **Respuestas:**
 - **200 OK:** Devuelve totales/listado y permite export. Ej.: `{ totals:{...}, items:[...] }`.

Módulo 2: Progresión y Reconocimientos

Caso: Registro de avance con evidencia

- Dado: un scouter autenticado y un joven con plan de adelanto
- Cuando: se marca una actividad como completada con evidencia adjunta
- Entonces: el sistema la registra como “pendiente de aval” con fecha y datos del scouter
- **Endpoint:** `POST /progress`
- **Respuestas:**
 - **201 Created:** Crea avance “pendiente de aval”. Ej.: `{ id, state:"pending_approval", createdAt }`.

Caso: Registro sin evidencia

- Dado: un scouter
- Cuando: intenta registrar un avance sin evidencia en una actividad obligatoria
- Entonces: el sistema muestra un error indicando que falta la evidencia
- **Endpoint:** `POST /progress`
- **Respuestas:**
 - **400/422 Bad Request:** Falta evidencia obligatoria. Ej.: `{ errors:[{ field:"evidence", message }] }`.

Caso: Aval de actividad por scouter

- Dado: una actividad marcada como “pendiente de aval”
- Cuando: el scouter responsable revisa y valida
- Entonces: cambia a estado “validada” y se guarda la trazabilidad (usuario, fecha, comentarios)
- **Endpoint:** `PUT /progress/{id}/approve`
- **Respuestas:**
 - **200 OK:** Cambia a “validada” con trazabilidad. Ej.: `{ id, state:"validated", audit:{ user, date, comment } }`.

Caso: Asignación de insignia tras validación

- Dado: un joven con actividades validadas
- Cuando: el scouter le asigna la insignia correspondiente

- Entonces: queda registrada con estado “Asignada” y aparece en su historial
- **Endpoint:** `POST /badges`
- **Respuestas:**
 - **201 Created: Registra insignia. Ej.:** `{ id, memberId, badgeId, state:"Assigned" }`.

Caso: Registro de ceremonia

- Dado: un jefe de grupo con eventos próximos
- Cuando: crea una ceremonia con fecha, tipo y participantes
- Entonces: la ceremonia queda registrada y vinculada a los jóvenes e insignias entregadas
- **Endpoint:** `POST /ceremonies`
- **Respuestas:**
 - **201 Created: Crea ceremonia vinculada. Ej.:** `{ id, date, type, participants:[...] }`.

Caso: Generación de reporte por rama (admin / acudiente)

- Dado: un usuario con rol de administrador
- Cuando: genera un reporte de progresión filtrando por rama y fecha
- Entonces: el sistema muestra las estadísticas e información exportable a PDF/Excel
- **Endpoint:** `GET /progress/reports?branch=&from=&to=`
- **Respuestas:**
 - **200 OK: Devuelve estadísticas/export. Ej.:** `{ stats:{...}, items:[...] }`.

Caso: Consulta de reporte de progresión por acudiente

- Dado: un acudiente autenticado
- Cuando: accede al reporte de su hijo
- Entonces: el sistema muestra las insignias completadas, en curso y pendientes, con la fecha del último avance y el scouter que lo avaló
- **Endpoint:** `GET /progress/my-child`
- **Respuestas:**
 - **200 OK: Muestra completadas/en curso/pendientes y última actualización. Ej.:** `{ childId, completed:[...], inProgress:[...], pending:[...], lastUpdate:{ date, scouter } }`.

Módulo 3: Eventos

Caso: Creación de evento con todos los datos

- Dado: un usuario con rol de Jefe de Grupo
- Cuando: completa todos los campos requeridos para crear un evento (nombre, fecha, lugar, responsables)
- Entonces: el evento queda registrado correctamente y aparece en el listado de eventos futuros
- **Endpoint:** `POST /events`
- **Respuestas:**
 - **201 Created: Crea evento en "draft". Ej.: { id, name, date, place, state:"draft" }.**

Caso: Creación de evento con datos faltantes

- Dado: un usuario autorizado
- Cuando: intenta registrar un evento sin ingresar información obligatoria
- Entonces: el sistema devuelve un error 400 indicando los campos incompletos
- **Endpoint:** `POST /events`
- **Respuestas:**
 - **400/422 Bad Request: Retorna errores de validación. Ej.: { errors:[{ field, message }] }.**

Caso: Asociación de miembros a un evento

- Dado: un evento ya creado
- Cuando: se asignan miembros participantes al evento (jóvenes y scouts)
- Entonces: la lista de participantes queda registrada correctamente en el evento
- **Endpoint:** `POST /events/{id}/participants`
- **Respuestas:**
 - **200 OK: Guarda participantes. Ej.: { eventId, participants:[{ memberId, role }] }**

Caso: Registro de autorización parental

- Dado: un miembro menor de edad asignado a un evento
- Cuando: su acudiente autoriza la participación desde su cuenta
- Entonces: la autorización queda registrada y visible para los responsables del evento
- **Endpoint:** `POST /events/{id}/consents`
- **Respuestas:**

- **201 Created: Registra consentimiento.** Ej.: {
memberId, approved:true, approvedBy, approvedAt }.

Caso: Check-in en evento

- Dado: un evento en curso
- Cuando: se realiza el check-in de un miembro desde la lista de participantes
- Entonces: se registra su asistencia con fecha y hora
- **Endpoint:** POST /events/{id}/checkin
- **Respuestas:**
 - **201 Created: Registra asistencia con fecha/hora.**
Ej.: { eventId, memberId, at }.

Caso: Control de aforo

- Dado: un evento con cupo máximo definido
- Cuando: el aforo se completa
- Entonces: el sistema no permite nuevas inscripciones y notifica al responsable
- **Endpoint:** POST /events/{id}/register
- **Respuestas:**
 - **201 Created: Inscribe cuando hay cupo.**
 - **409 Conflict: No inscribe; aforo completo.**

Caso: Consulta de lista médica

- Dado: un jefe de grupo con un evento activo
- Cuando: consulta la lista médica de los participantes
- Entonces: el sistema muestra la información sensible vinculada (EPS, alergias, contactos) solo a usuarios autorizados
- **Endpoint:** GET /events/{id}/medical-list
- **Respuestas:**
 - **200 OK: Devuelve info médica solo a roles autorizados.** Ej.: [{ memberId, eps, allergies, emergencyContact }].

Módulo 4: Finanzas

Caso: Registro de cuota por familia

- Dado: un usuario con rol de Gestor Administrativo
- Cuando: registra una cuota de sostenimiento para una familia

- Entonces: el sistema guarda el monto, fecha y estado de pago asociado a los miembros de la familia
- **Endpoint:** `POST /fees`
- **Respuestas:**
 - **201 Created:** Crea cuota. Ej.: `{ id, familyId, amount, period, status:"pending" }`

Caso: Registro de aporte adicional

- Dado: una familia o miembro que realiza un aporte extra
- Cuando: el administrador registra el monto como “aporte adicional”
- Entonces: el aporte queda identificado como tal en los reportes y asociado al contribuyente
- **Endpoint:** `POST /donations`
- **Respuestas:**
 - **201 Created:** Registra aporte. Ej.: `{ id, contributorId, amount, date, type:"donation" }.`

Caso: Registro de pago parcial

- Dado: una cuota pendiente por parte de una familia
- Cuando: se recibe un abono parcial
- Entonces: el sistema registra el monto pagado y muestra el saldo restante
- **Endpoint:** `POST /payments`
- **Respuestas:**
 - **201 Created:** Registra abono y saldo. Ej.: `{ paymentId, feeId, amount, remaining }.`

Caso: Pago completo de cuota

- Dado: una cuota activa
- Cuando: el usuario realiza el pago total
- Entonces: la cuota cambia de estado a “pagada” y se actualiza la contabilidad general
- **Endpoint:** `POST /payments`
- **Respuestas:**
 - **01 Created:** Marca la cuota como pagada. Ej.: `{ paymentId, feeId, remaining:0, feeStatus:"paid" }.`

Caso: Consulta de estado de cuenta

- Dado: un acudiente autenticado
- Cuando: accede a su sección de finanzas

- Entonces: puede ver las cuotas pendientes, abonadas y pagadas con fechas y descripciones
- **Endpoint:** GET /families/me/billing
- **Respuestas:**
 - **200 OK: Muestra pendientes/abonadas/pagadas. Ej.: { familyId, totals:{ pending, partial, paid }, items:[...] }**

Caso: Generación de reporte financiero

- Dado: un Gestor Administrativo
- Cuando: genera un reporte por periodo (ej: mes o trimestre)
- Entonces: el sistema entrega un documento con totales por categoría (cuotas, aportes, deudas), exportable en PDF/Excel
- **Endpoint:** GET /finance/reports?from=&to=
- **Respuestas:**
 - **200 OK: Devuelve totales por categoría y permite export.**

Caso: Notificación automática de pagos pendientes

- Dado: una familia con cuotas vencidas
- Cuando: llega la fecha límite sin pago
- Entonces: el sistema envía un recordatorio automático por el canal configurado (correo) y registra el envío en el log de notificaciones
- **Endpoint:** POST /finance/notifications/run
- **Respuestas:**
 - **202 Accepted: Programa/envía recordatorios y registra log. Ej.: { jobId, scheduledAt }.**

Colecciones Postman APIs mock

Para estructurar las pruebas de API en **Postman Desktop**, QA mantendrá **una colección por módulo**.

1. Organización de Colecciones

- **Colección: Login**
 - Endpoint: `POST /auth/login`
 - Requests: credenciales válidas, credenciales inválidas, tenant incorrecto.
- **Colección: Miembros**
 - Endpoints: `GET /members`, `POST /members`, `PUT /members/{id}`, `DELETE /members/{id}`
- **Colección: Eventos**
 - Endpoints: `GET /events`, `POST /events`, `PUT /events/{id}/authorize`
- **Colección: Roles/Permisos**
 - Endpoints: `GET /roles`, `GET /permissions`

2. Variables de Entorno

Se usará un **Environment por tenant** para aislar pruebas:

Ejemplo de Environment **grupo1**:

- `baseUrl` → `https://mockapi.io/grupo1`
- `tenantSlug` → `grupo1`
- `authToken` → (se guarda dinámicamente después del login válido)

Ejemplo de Environment **grupo2**:

- `baseUrl` → `https://mockapi.io/grupo2`
- `tenantSlug` → `grupo2`
- `authToken` → (se actualiza automáticamente con el login de grupo2)

Beneficio: Al cambiar de Environment en Postman, todas las requests apuntan al tenant correspondiente sin duplicar pruebas.

3. Configuración de Tests en Postman (sin código manual)

En cada request se configurarán **Tests desde la pestaña de Tests de Postman Desktop**:

- **Status Code:** verificar que la respuesta sea **200 OK** (o el esperado).
- **Validación de tenant:** asegurar que el **tenant_id** devuelto coincide con la variable de entorno **{{tenantSlug}}**.
- **Esquema de respuesta:** confirmar que los campos mínimos (ej. **id**, **name**, **createdAt**) están presentes.
- **Autenticación:** en Login, guardar automáticamente el token (**authToken**) en el Environment para reutilizarlo en las demás requests.

4. Buenas prácticas en Postman

- **Agrupar requests por carpeta dentro de cada colección** (ej. “Miembros – Crear”, “Miembros – Listar”).
- **Describir cada request con lenguaje natural** (ej. “*Crear miembro válido*”, “*Login inválido*”).
- **Usar ejemplos (Examples)** en Postman para documentar la request/response esperada.
- **Exportar colecciones y environments** en JSON para compartirlos con el equipo.

Smoke Suite (para CI/CD)

Caso: Iniciar sesión

- **Dado:** un usuario (líder, miembro, o comité) registrado con credenciales válidas
- **Cuando:** intenta iniciar sesión con su nombre de usuario y contraseña
- **Entonces:** el sistema autentica al usuario y redirige al dashboard correspondiente
- **Endpoint:** POST /login
- **Respuestas:**
 - **200 OK:** Inicio de sesión exitoso. Ej.: { token: <>, user_id: <>, role: <> }.
 - **500 error:** Credenciales inválidas.

Caso: Cerrar sesión

- **Dado:** un usuario (líder, miembro, o comité) autenticado en el sistema
- **Cuando:** selecciona la opción de cerrar sesión
- **Entonces:** el sistema cierra la sesión y redirige al usuario a la página de inicio de sesión
- **Endpoint:** POST /logout
- **Respuestas:**
 - **200 OK:** Cierre de sesión exitoso. Ej.: { mensaje: "Sesión cerrada correctamente" }.
 - **401 Unauthorized:** No hay sesión activa.

Caso: Consulta de miembros

- **Dado:** un usuario autorizado
- **Cuando:** realiza una búsqueda filtrando por nombre, rama, rol o estado
- **Entonces:** el sistema devuelve únicamente los miembros que cumplen el criterio, mostrando información básica y opción de detalle
- **Endpoint:** GET /members?name=&branch=&role=&status=
- **Respuestas:**
 - **200 OK:** Devuelve lista filtrada de miembros del tenant. Ej.: [{ id, name, branch, role, status }].

Caso: Registro de miembro válido

- **Dado:** un usuario con permisos de Jefe de Grupo
- **Cuando:** registra un nuevo miembro con todos los campos requeridos (nombre, edad, acudiente, rol)
- **Entonces:** el miembro se guarda correctamente y aparece en el listado actualizado
- **Endpoint:** POST /members
- **Respuestas:**

- **201 Created:** Crea el miembro. Ej.: { id, name, age, guardian, role, tenant_id }.

Caso: Registro de miembro con datos incompletos

- Dado: un usuario autorizado
- Cuando: intenta registrar un nuevo miembro sin completar campos obligatorios
- Entonces: el sistema devuelve un error 400 con un mensaje indicando los campos faltantes
- **Endpoint:** POST /members
- **Respuestas:**
 - **400/422 Bad Request:** Devuelve errores por campo. Ej.: { errors:[{ field, message }] }.

Caso: Eliminación lógica o baja de un miembro

- Dado: un miembro activo
- Cuando: el usuario con permisos lo desactiva
- Entonces: el miembro deja de aparecer en el listado pero se conserva en el historial
- **Endpoint:** DELETE /members/{id}
- **Respuestas:**
 - **204 No Content:** El miembro queda en historial y no aparece en listados.

Caso: Listar eventos

- **Dado:** un usuario líder o comité
- **Cuando:** solicita ver la lista de eventos de su grupo
- **Entonces:** el sistema muestra una lista de todos los eventos registrados en el grupo
- **Endpoint:** GET /events
- **Respuestas:**
 - **200 OK:** Lista de objetos tipo evento.
 - **403 Forbidden:** El usuario no tiene permisos para acceder a esta información.

Caso: Autorizar evento

- **Dado:** un usuario líder o comité
- **Cuando:** autoriza un evento existente
- **Entonces:** el evento se marca como autorizado y se notifica a los participantes
- **Endpoint:** POST /events/{id}/authorize

- **Respuestas:**
- **200 OK:** Evento autorizado. Ej.: { mensaje: "Evento autorizado correctamente" }.
- **404 Not Found:** El evento no existe.
- **403 Forbidden:** El usuario no tiene permisos para autorizar eventos.

Caso: Desautorizar evento

- **Dado:** un usuario líder o comité
- **Cuando:** desautoriza un evento previamente autorizado
- **Entonces:** el evento se marca como no autorizado y se notifica a los participantes
- **Endpoint:** `POST /events/{id}/unauthorize`
- **Respuestas:**
- **200 OK:** Evento desautorizado. Ej.: { mensaje: "Evento desautorizado correctamente" }.
- **404 Not Found:** El evento no existe.
- **403 Forbidden:** El usuario no tiene permisos para desautorizar eventos.

Seguridad y Aislamiento Multi-tenant

1. Casos de Prueba de Aislamiento Multi-tenant

Caso 1.1: Acceso cruzado entre grupos - Lectura de miembros

Prioridad: P1 (Crítica) **Precondición:**

- Usuario `jefe_grupo1` autenticado con token válido del Grupo Scout A (tenant: grupo1)
- Existe al menos un miembro registrado en Grupo Scout B (tenant: grupo2)

Pasos:

1. Autenticarse como `jefe_grupo1` en `/auth/login`
2. Intentar acceder a `GET /grupo2/members`
3. Alternativamente, intentar `GET /members` con header `X-Tenant-Id: grupo2`

Resultado esperado:

- Status: 403 Forbidden o 404 Not Found
- Respuesta: `{"error": "Acceso denegado"}`
- No debe retornar datos del grupo2

Caso 1.2: Manipulación de `tenant_id` en requests

Prioridad: P1 (Crítica) **Precondición:** Usuario autenticado del grupo1

Pasos:

1. Autenticarse como usuario del grupo1
2. Hacer `POST /members` con body que incluya `"tenant_id": "grupo2"`
3. Verificar en base de datos donde se creó el registro

Resultado esperado:

- El sistema debe ignorar el `tenant_id` del body
- El miembro debe crearse en grupo1 (tenant del usuario autenticado)
- O rechazar la request con error 400

Caso 1.3: Acceso a eventos de otro grupo

Prioridad: P1 (Crítica)

Pasos:

1. Crear evento en grupo1 y obtener su ID
2. Autenticarse como usuario del grupo2
3. Intentar `GET /events/{id_evento_grupo1}`
4. Intentar `POST /events/{id_evento_grupo1}/authorize`

Resultado esperado:

- Status: 404 Not Found
- No debe permitir ninguna operación sobre recursos de otro tenant

2. Casos de Validación de Datos Sensibles por Rol

Caso 2.1: Rol "miembro" no debe ver datos médicos

Prioridad: P1 (Crítica) **Precondición:** Usuario con rol "miembro" autenticado

Pasos:

1. Autenticarse como miembro regular
2. Hacer `GET /members/{id}` de otro miembro del mismo grupo
3. Revisar campos en la respuesta

Resultado esperado:

```
json
{
  "id": 123,
  "name": "Juan Pérez",
  "branch": "Tropa",
  "role": "Scout"
  // NO deben aparecer: eps, allergies, medications, blood_type
}
```

Caso 2.2: Rol "acudiente" solo ve datos de sus hijos

Prioridad: P1 (Crítica)

Pasos:

1. Autenticarse como acudiente
2. Hacer `GET /members`
3. Hacer `GET /members/{id_hijo}`
4. Hacer `GET /members/{id_otro_joven}`

Resultado esperado:

- Lista de miembros: solo muestra sus hijos
- Detalle de hijo: incluye datos médicos
- Detalle de otro joven: 403 Forbidden

Caso 2.3: Rol "jefe" ve datos sensibles con restricciones

Prioridad: P2 (Media)

Pasos:

1. Autenticarse como jefe de rama
2. Hacer `GET /events/{id}/medical-list`
3. Verificar campos retornados

Resultado esperado:

json

```
{
  "members": [
    {
      "id": 123,
      "name": "Juan Pérez",
      "eps": "Sura",
      "allergies": ["Polen"],
      "emergency_contact": "3001234567"
      // NO debe incluir: documentos, direcciones completas
    }
  ]
}
```

3. Validación HTTPS

Caso 3.1: Redirección HTTP a HTTPS

Prioridad: P1 (Crítica) **Ambiente:** Staging/Producción

Pasos:

1. Intentar acceder a <http://staging.plataformascout.org/login>
2. Verificar redirección

Resultado esperado:

- Redirección 301 a <https://staging.plataformascout.org/login>
- Header `Strict-Transport-Security` presente

Caso 3.2: Validación de certificado SSL

Prioridad: P2 (Media)

Verificaciones:

- Certificado válido y no expirado
- Certificado emitido para el dominio correcto
- Cadena de certificación completa
- TLS 1.2 o superior